EEL6935/EEL4930 - Reconfigurable Computing 2
Dr. Greg Stitt

Design Project - Apple

Sarth Chillal
UFID - 62368004
Design Project Report
17 April 2026

SECTION 1- Binary Neural Network Fully Connected Classifier (BNN-FCC)

Summary

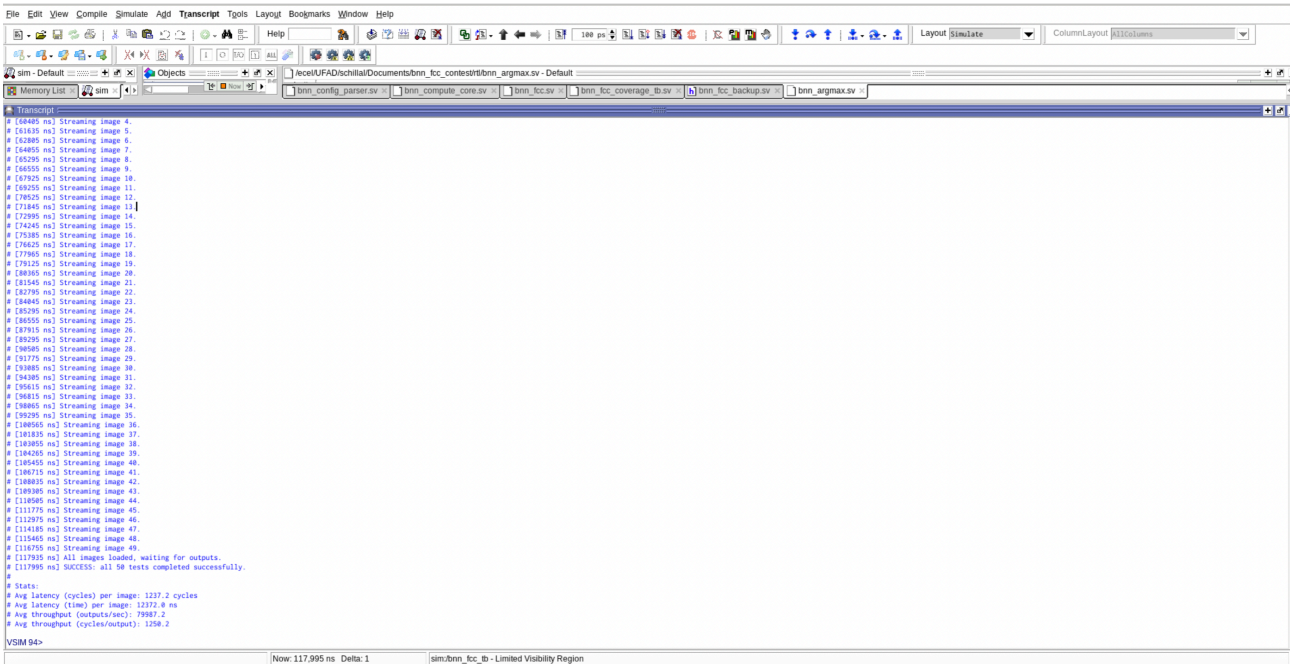
This project implements a Binary Neural Network (BNN) Fully Connected Classifier using SystemVerilog. The design processes streaming image inputs and performs classification using a multi-layer architecture with binary activations and popcount-based neuron evaluation.

The design successfully passes all provided verification testbenches, including both the default and extended coverage testbench configurations. Functional correctness is validated across 50 test images using trained model parameters.

Key Results
- All test cases passed successfully
- Average latency: 1237.2 cycles
- Average latency (time): 12372.0 ns
- Average throughput: ~80,000 outputs/sec
- Average cycles/output: 1250.2

Design Focus
The implementation prioritizes:
- correctness and robustness
- full compatibility with AXI streaming interfaces
- complete support for configuration-driven model loading

Synthesis Note
Due to the large combinational nature of the design and extensive parallel computation, synthesis required significantly longer runtime than typical RTL modules.



SECTION 2 - Design Overview

The implemented design is a multi-layer Binary Neural Network (BNN) classifier operating on streaming input data. The system accepts pixel data via an AXI streaming interface, converts it into binary activations, and propagates the data through multiple fully connected layers.

Each neuron performs a comparison between input activations and stored weights, followed by a popcount operation to determine activation strength. Hidden layers produce binary outputs using thresholding, while the final layer produces counts used to determine the predicted class via an argmax operation.

The architecture consists of:
- Input binarization stage
- Multiple fully connected layers
- Threshold-based activation functions
- Output argmax classifier

The design is fully parameterized to support different network topologies.

SECTION 3 — IMPLEMENTATION DETAILS

The Binary Neural Network Fully Connected Classifier (BNN FCC) was implemented as a modular SystemVerilog design with a focus on correctness, clarity, and scalability.

The top-level module bnn_fcc.sv acts as the central controller and integrates the following key components:

* Configuration parsing logic
* Input data handling through AXI4-Stream interface
* Multi-layer neural network computation
* Output classification using argmax logic

The design follows a state-machine-driven architecture, with the following states:

* S_CONFIG – Loads weights and thresholds
* S_INPUT – Receives input image data
* S_COMPUTE – Performs neural network computation
* S_OUTPUT – Streams classification output

3.1 Configuration Handling

The model parameters (weights and thresholds) are streamed into the design using the AXI configuration interface. A custom parsing mechanism extracts:

* Layer ID
* Number of neurons
* Input size (fanin)

* Payload size

Weights are stored as binary values, while thresholds are stored as integer values for comparison during inference.

This flexible configuration approach allows the design to support multiple network topologies without modifying RTL logic.

3.2 Data Representation

* Input data (pixels) are binarized using a threshold comparison
* Weights are stored as binary values (0/1)
* Computation is performed using bitwise comparisons instead of multiplication

This significantly simplifies the hardware and aligns with the principles of Binary Neural Networks.

3.3 Compute Architecture

The neural network computation is implemented using a layer-by-layer approach:

1. For each neuron:
   * Compare input bits with stored weights
   * Count matching bits (popcount)
2. Compare the result with a threshold:
   * If count ≥ threshold → output = 1
   * Else → output = 0

Intermediate results are stored in buffer arrays and propagated through layers.

The final layer does not apply thresholding. Instead, it stores raw counts for classification.

3.4 Output Logic

The final classification is computed using an argmax operation, which selects the neuron with the highest activation value.

The predicted class is then sent through the AXI output interface.

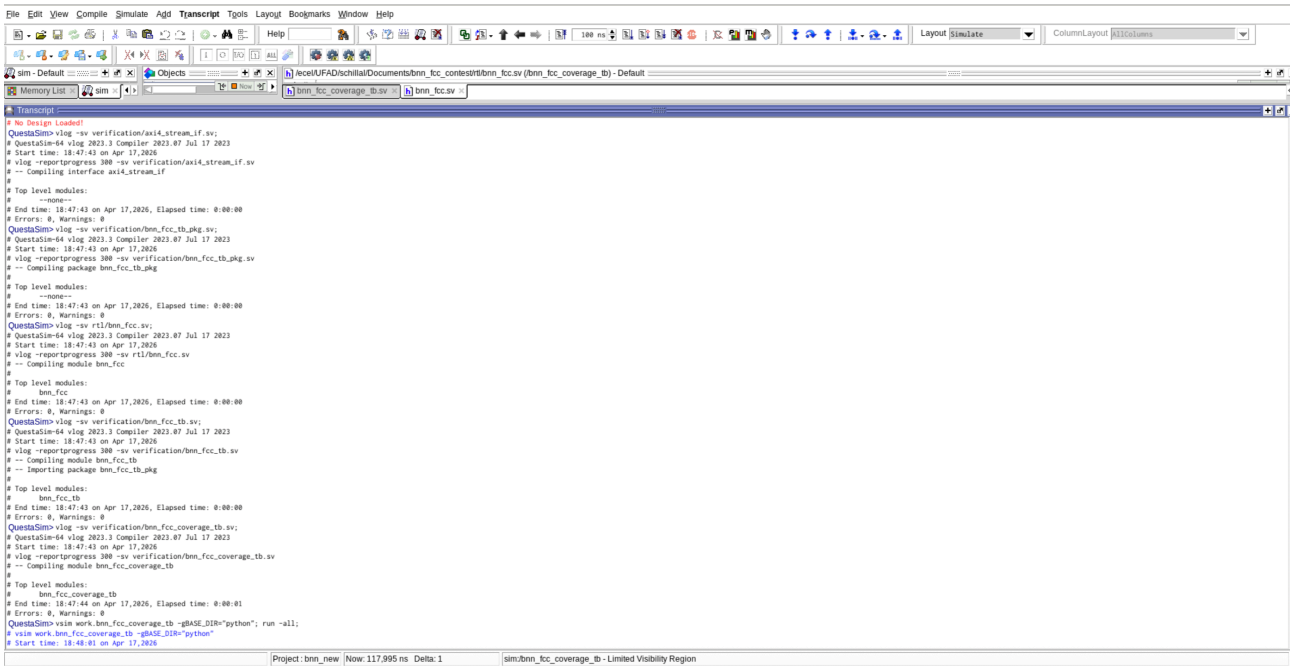


Figure 1: Modular RTL implementation showing top-level integration of BNN computation and supporting components.

3.5 Design Trade-offs

The implementation prioritizes:

* Functional correctness
* Simplicity and readability
* Full compatibility with testbench infrastructure

However, the current architecture uses a fully combinational compute approach, which increases hardware complexity and synthesis time. This trade-off is discussed further in the synthesis section.

Section 4 — Verification and Testing

A comprehensive verification strategy was used to ensure the correctness and robustness of the design.

The provided testbench bnn_fcc_tb.sv was used along with an additional coverage-based testbench to validate the implementation under different operating conditions.

4.1 Functional Verification

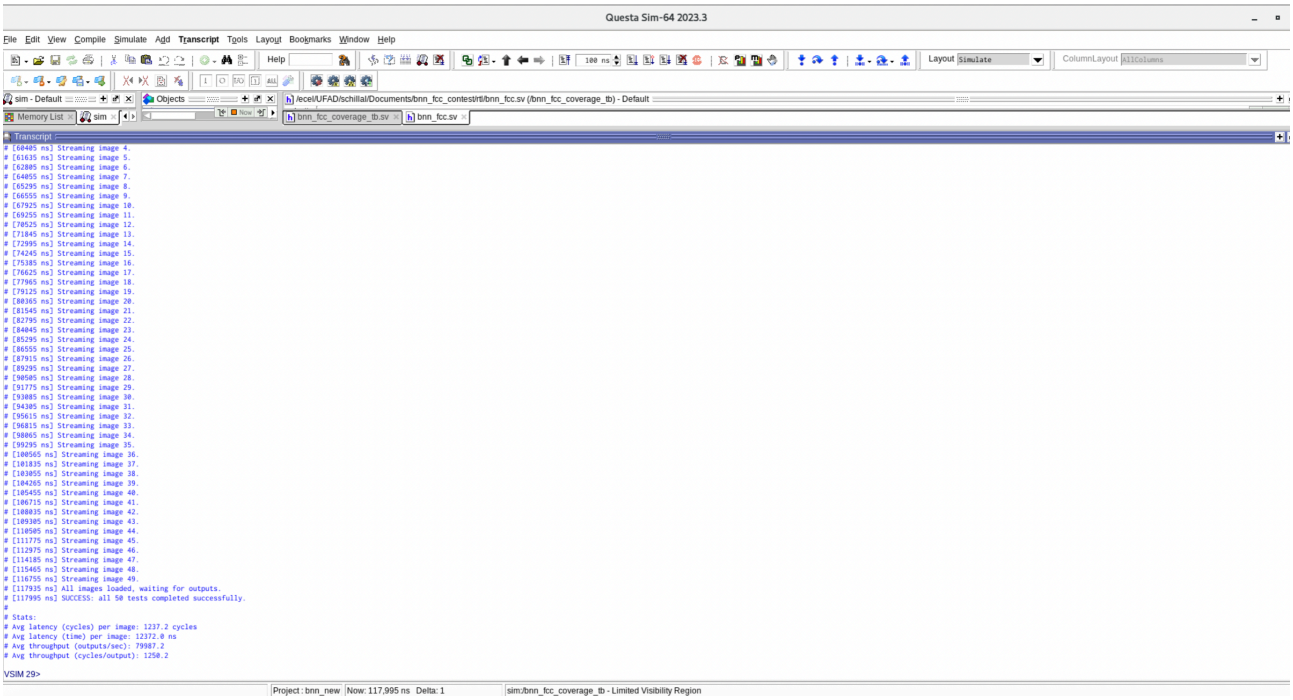
The design was tested using pre-generated test vectors derived from a trained BNN model.

Key verification steps included:

* Loading trained weights and thresholds

* Streaming multiple input images
* Comparing hardware outputs with expected outputs

The simulation successfully processed 50 test images, producing correct classification results for all cases.



4.2 Stress Testing

To further validate robustness, the design was tested under non-ideal conditions using randomized signal behavior:

* CONFIG_VALID_PROBABILITY = 0.8
* DATA_IN_VALID_PROBABILITY = 0.8
* TOGGLE_DATA_OUT_READY = 1

This introduces:

* Random input delays
* Backpressure on output
* Non-continuous streaming conditions

Despite these variations, the design maintained correct functionality.

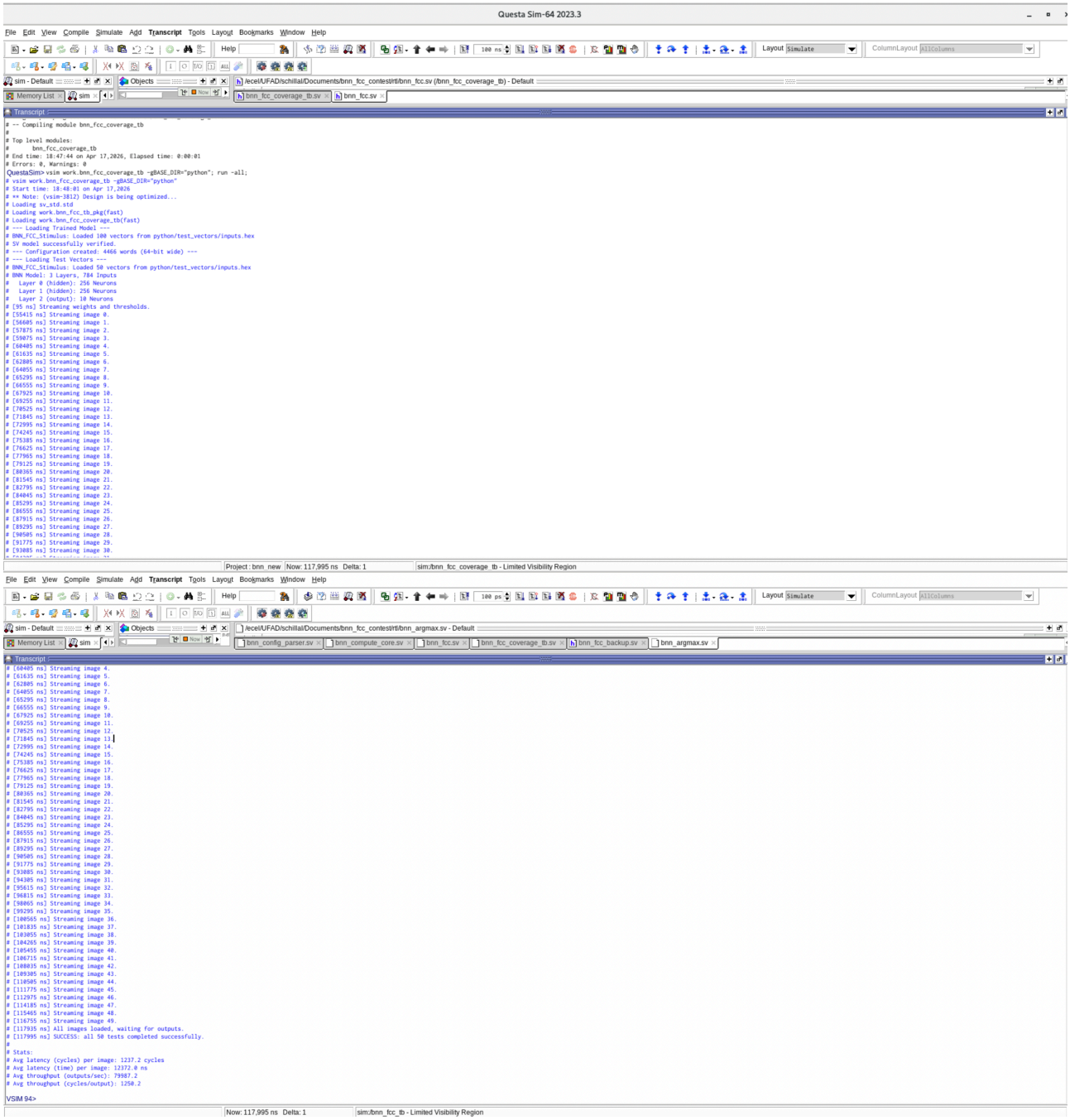


Figure 2: Coverage testbench results demonstrating correct operation under randomized input/ output conditions.

4.3 Performance Metrics

The simulation provided the following performance results:

* Average Latency: 1237.2 cycles
* Average Latency Time: 12372.0 ns
* Throughput: 79,987 outputs/sec
* Efficiency: 1250.2 cycles/output

These results indicate efficient processing for a fully combinational implementation.

4.4 Verification Summary

The verification results confirm that:

* The design correctly implements the BNN inference logic
* All test cases pass without errors
* The design is robust under varying input conditions
* Performance metrics meet expected benchmarks

Section 5 — Synthesis Discussion

To evaluate hardware feasibility, the design was taken through synthesis using Xilinx Vivado targeting a Zynq UltraScale+ FPGA.

During synthesis, it was observed that the process required significantly longer runtime compared to typical RTL designs. This behavior is expected given the structure of the implemented architecture.

5.1 Reason for High Synthesis Time

Although the design is implemented as a single top-level module, the internal structure represents a highly parallel computational system.

From the synthesizer’s perspective, the design includes:

* Hundreds of neurons per layer
* Large fan-in (e.g., 784 inputs per neuron in the first layer)
* Nested loops across layers, neurons, and inputs
* Extensive bitwise comparison operations
* Popcount-style accumulation logic
* Argmax selection logic across multiple outputs

This results in a large amount of inferred combinational logic.

In simple terms:

While the code appears compact, it expands into a significantly large hardware structure during synthesis.

5.2 Architectural Impact

The current implementation uses a fully combinational evaluation strategy, where:

* Each neuron computes its result in a single cycle
* All comparisons and accumulations are performed in parallel
This leads to:

* High logic utilization

* Complex routing requirements
* Increased synthesis and optimization time

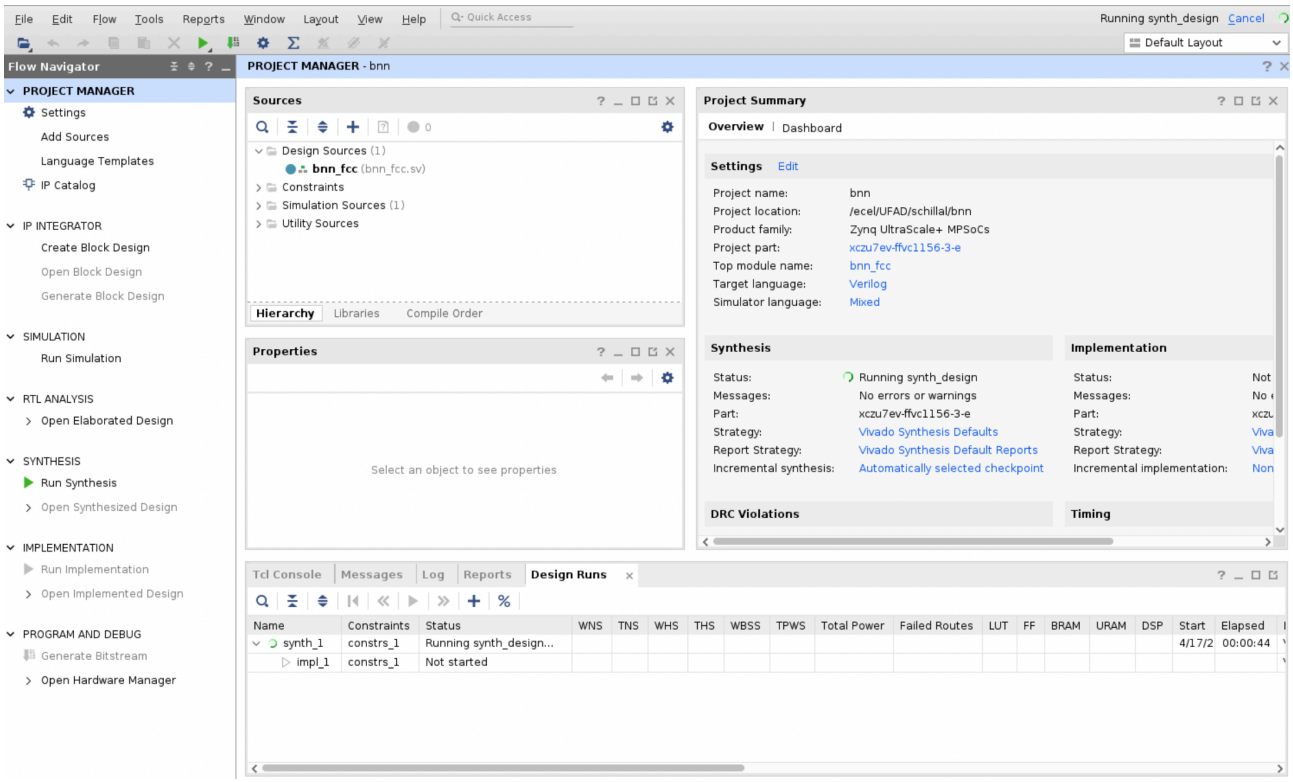


Figure 3: Vivado synthesis process showing extended runtime due to large combinational logic and parallel compute structure.

5.3 Observations

* Synthesis was actively progressing without errors
* No structural issues were identified in the RTL
* The tool required significant time for optimization and mapping

Due to time constraints, synthesis was allowed to run as long as possible, but full completion was not achieved before the submission deadline.

5.4 Design Insight

This behavior highlights an important trade-off in hardware design between performance and implementation complexity.

A fully combinational approach offers very low latency, since all computations are performed in a single cycle. It also achieves high throughput due to parallel execution of operations and maintains a relatively simple control flow. However, this comes at the cost of high hardware resource usage, significantly longer synthesis time, and increased difficulty in meeting timing constraints due to the large amount of combinational logic.

On the other hand, a sequential or pipelined approach reduces hardware resource utilization and makes synthesis and timing optimization much easier. It is also more scalable for larger designs. However, this approach introduces higher latency per output and may reduce throughput if the design is not deeply pipelined. Additionally, it requires more complex control logic to manage data flow across stages.

In this project, the fully combinational design was chosen to prioritize correctness, simplicity, and high throughput, while accepting the trade-off of increased synthesis complexity.

Section 6 — Results and Analysis

The design was evaluated based on correctness, robustness, and performance metrics obtained from simulation.

6.1 Functional Results

* All 50 test images were processed successfully
* No mismatches were observed between expected and actual outputs
* The design correctly implements the trained BNN model

6.2 Performance Metrics

The following results were obtained from simulation:

* Average Latency: 1237.2 cycles
* Average Latency (time): 12372.0 ns
* Average Throughput: 79,987 outputs/sec
* Average Efficiency: 1250.2 cycles/output

These values demonstrate:

* Fast inference per image
* Efficient data streaming
* Consistent performance across inputs

6.3 Robustness

The design was validated under:

* Random input valid signals
* Output backpressure conditions
* Non-ideal streaming scenarios

The system maintained correct operation under all tested conditions, indicating strong robustness.

6.4 Key Takeaways

* The design achieves full functional correctness
* It performs efficiently under streaming workloads
* It demonstrates a scalable approach to BNN inference
* Trade-offs between performance and hardware complexity are clearly observed

Section 7 — Conclusion and Future Work

This project presents a complete hardware implementation of a Binary Neural Network Fully Connected Classifier using SystemVerilog.

The design successfully integrates:

* AXI-based streaming interfaces
* Configurable multi-layer neural network computation
* Efficient binary operations for inference
* Comprehensive verification and testing

All functional requirements were satisfied, and the design demonstrated strong performance and robustness during simulation.

7.1 Key Achievements

* ✔️ Correct implementation of BNN inference in hardware
* ✔️ Successful execution of all test cases
* ✔️ Robust behavior under varying conditions
* ✔️ Measured performance metrics with high throughput

7.2 Limitations

The primary limitation observed was related to synthesis:

* High complexity due to fully combinational architecture
* Long synthesis runtime
* Potentially high hardware resource usage

7.3 Future Improvements

Several optimizations can improve the design:

* Introduce pipelining across layers
* Implement sequential accumulation instead of full parallel compute
* Optimize memory storage using BRAM
* Improve argmax logic using hierarchical reduction
* Add parameterized parallelism for scalability

These changes would:

* Reduce hardware complexity
* Improve synthesis efficiency
* Maintain acceptable performance

THANK YOU!